Task 1: Build and Inspect a View Hierarchy

```
import Foundation

class MyView {
    var name: String
    var x: Double
    var y: Double
    var width: Double
    var height: Double
    var subviews: [MyView] = []

    init(_ name: String, _ x: Double, _ y: Double, _ width: Double, _ height: Double) {
        self.name = name
        self.x = x
        self.y = y
        self.width = width
        self.height = height
    }

    func addSubview(_ view: MyView) {
        subviews.append(view)
    }

    func bringSubviewToFront(_ view: MyView) {
        subviews.removeAll { $0 === view }
```

```
        subviews.append(view)
    }

    func printHierarchy(_ level: Int = 0) {
        print(String(repeating: " ", count: level * 2) + name)
        for child in subviews {
            child.printHierarchy(level + 1)
        }
    }
}

let mainView = MyView("Main View", 0, 0, 400, 800)
let card = MyView("Teal Card", 40, 120, 280, 200)
let red = MyView("Red View", 20, 20, 100, 100)
let yellow = MyView("Yellow View", 70, 70, 100, 100)

mainView.addSubview(card)
card.addSubview(red)
card.addSubview(yellow)

print("View Hierarchy:")
mainView.printHierarchy()

print("\nDrawing order: Yellow View is on top")

card.bringSubviewToFront(red)
```

```
print("After bringing red to front:")
mainView.printHierarchy()

let originX = card.x + red.x
let originY = card.y + red.y

print("\nRed view origin in main view: (\(originX), \(originY))")
```



Task 2: Create a Custom UIView — BadgeView

```
import Foundation

struct BadgeView {
    var text: String
```

```
    var color: String
    var horizontalPadding: Int = 12
    var verticalPadding: Int = 6

    func display() {
        print("Badge Text: \(text)")
        print("Badge Color: \(color)")
        print("Horizontal Padding: \(horizontalPadding) points")
        print("Vertical Padding: \(verticalPadding) points")
        print("Style: Rounded corners")
    }
}

let badge = BadgeView(text: "NEW", color: "Blue")
badge.display()

print("Position: Center of screen")
```

Output

```
Badge Text: NEW
Badge Color: Blue
Horizontal Padding: 12 points
Vertical Padding: 6 points
Style: Rounded corners
Position: Center of screen
```

Task 3: Experiment — Frame versus Bounds

```
import Foundation

struct MyView {
    var x: Double
    var y: Double
    var width: Double
    var height: Double

    func printFrame() {
        print(String(format: "Frame: (x: %.2f, y: %.2f, width: %.2f, height: %.2f)", x, y, width, height))
    }

    func printBounds() {
        print(String(format: "Bounds: (x: 0.00, y: 0.00, width: %.2f, height: %.2f)", width, height))
    }

    mutating func rotate45Degrees() {
        let radians = Double.pi / 4
        let newWidth = abs(width * cos(radians)) + abs(height * sin(radians))
        let newHeight = abs(width * sin(radians)) + abs(height * cos(radians))

        x -= (newWidth - width) / 2
        y -= (newHeight - height) / 2
```

```
        width = newWidth
        height = newHeight
    }
}

var box = MyView(x: 50, y: 100, width: 100, height: 100)

print("Before rotation:")
box.printFrame()
box.printBounds()

box.rotate45Degrees()

print("\nAfter rotation:")
box.printFrame()
box.printBounds()
```

Output

```
Before rotation:
Frame: (x: 50.00, y: 100.00, width: 100.00, height: 100.00)
Bounds: (x: 0.00, y: 0.00, width: 100.00, height: 100.00)

After rotation:
Frame: (x: 29.29, y: 79.29, width: 141.42, height: 141.42)
Bounds: (x: 0.00, y: 0.00, width: 141.42, height: 141.42)
```